CM3070 Computer Science Final Year Project

Machine Learning and Neural Networks - Deep Learning Breast Cancer Detection

Done by: Aesterin Huang Qiaorui

Project Background & Context

The project template chosen is **Machine Learning and Neural Networks - Deep Learning Breast Cancer Detection.**

Project Background & Context

In early 2025, the UK National Health Service (NHS) launched a major project that addresses the question on whether Deep Learning (DL) assisted X-ray mammography is reliable enough to achieve accurate results or improve the accuracy of breast cancer screening.

The research foresees that a successful outcome of this project can potentially have a DL system to replace at least two radiologists for faster diagnostic turn-around and the liberation of specialists for other tasks.

Project Motivation

The motivation of attempting this project comes from the fact that breast cancer is the most common disease affecting mainly women. Research has shown that breast cancer generally has a lower mortality rate due to early awareness and detection. This highlights the critical role of investment in advanced mammography technology for improving screening accuracy and early diagnosis. Through this project, not only can we discover a potential possibility to change the trajectory of advanced cancer screening, but this research also has the potential to optimise healthcare and staff resource, alleviate the healthcare system's burnout, allow specialist to redirected towards higher priority clinical tasks and subsequently improve the overall efficiency of the healthcare system.

Problem Statement

Can Deep Learning assisted X-ray mammography improve the accuracy of breast cancer screening, on predicting whether the breast tumours in question are benign or malignant?

Target audience? Who will benefit this?

Radiologist and medical professionals responsible for handling and interpreting mammograms



The scope does not extend to actual clinical deployment or any integration with existing hospital mammography practices or systems.

The model will be developed and evaluated in a local environment without access to dedicated GPUs which may impose constrains on the model complexity.

History of mammography and mammogram

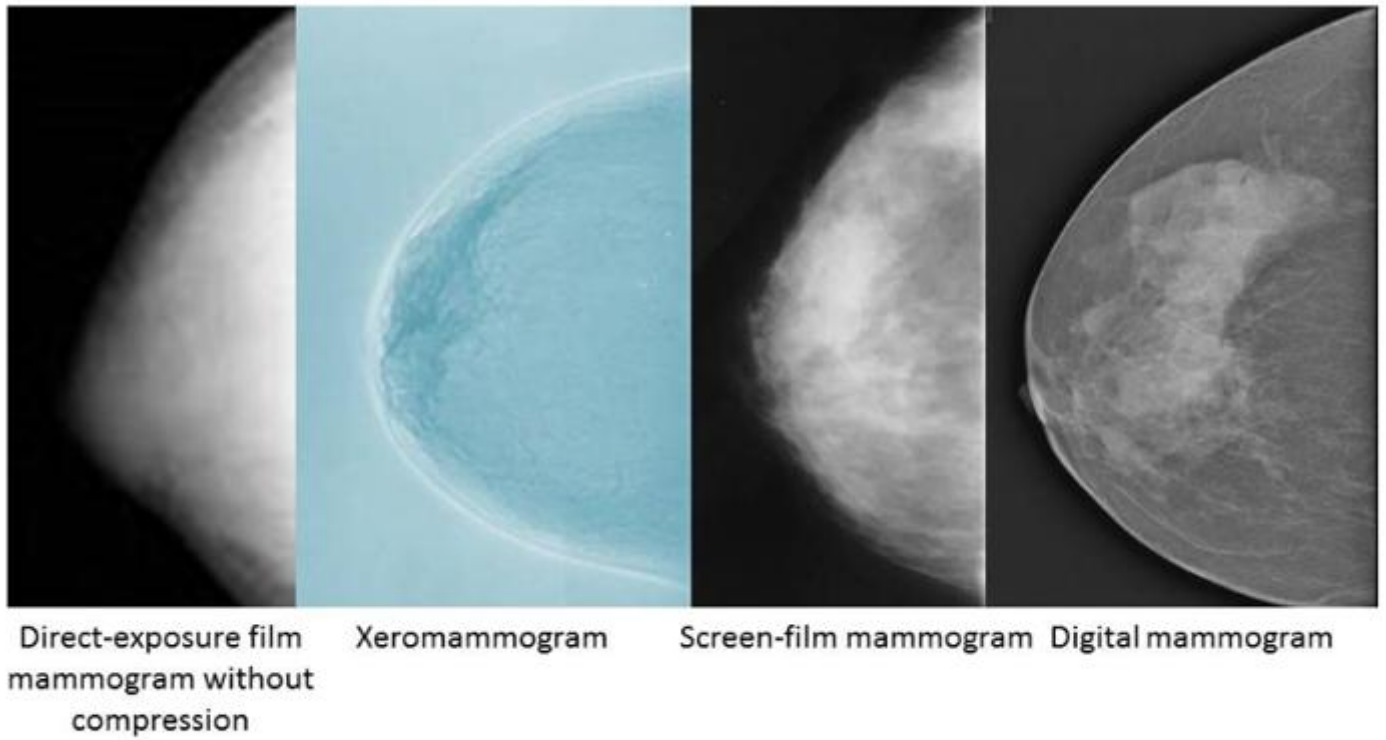
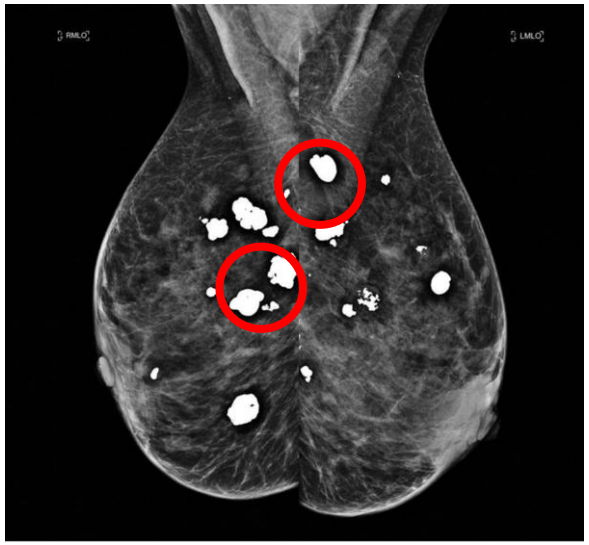
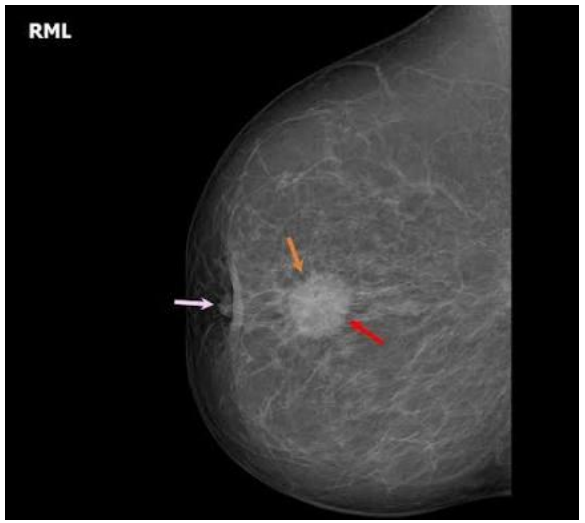


Figure 4 - The evolution of breast imaging techniques from 1965 to the present (n.d.)

Primary indicators/features



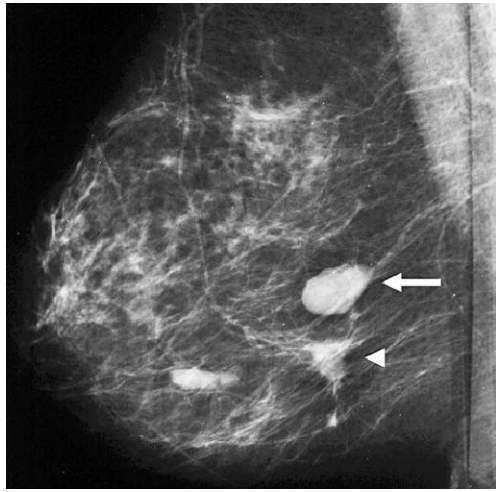
Calcification



Lumps/masses

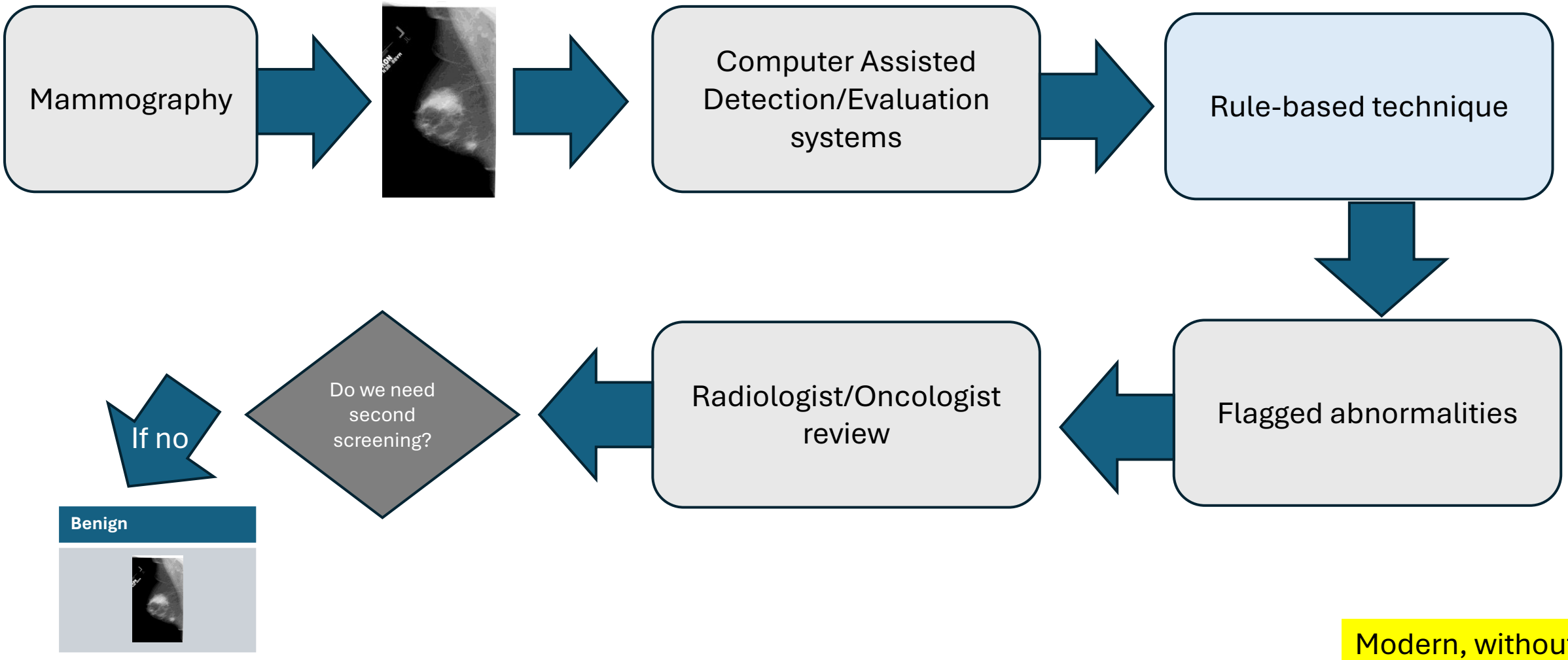


Different density/asymmetry



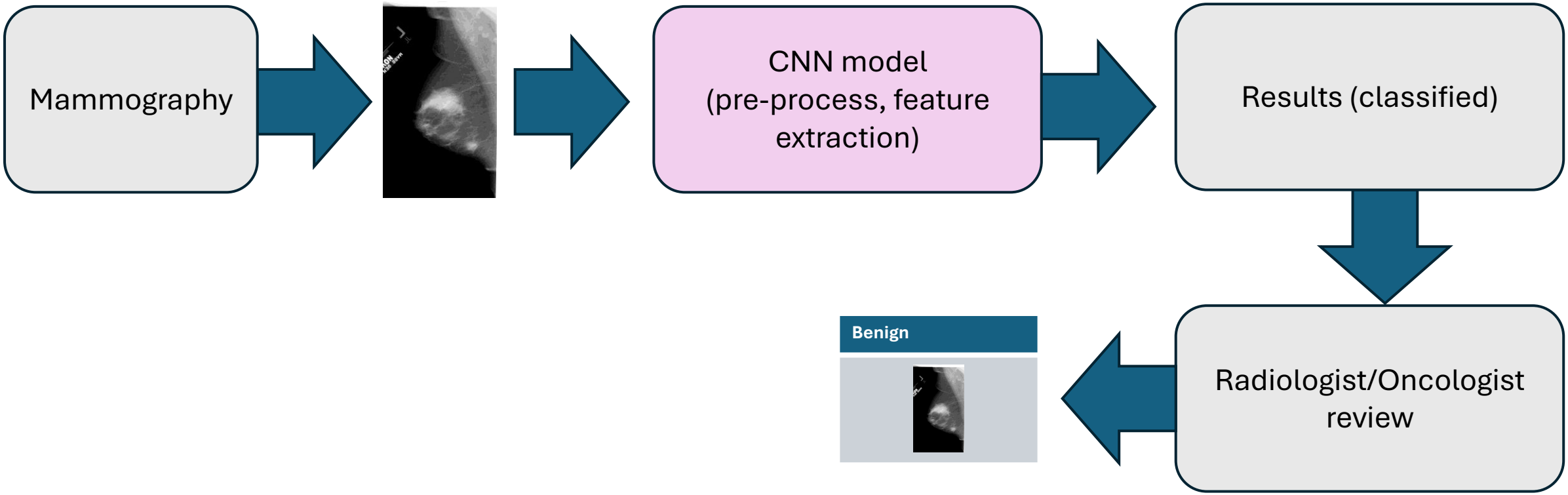
Malignant growth

How does Deep Learning help?



Modern, without DL

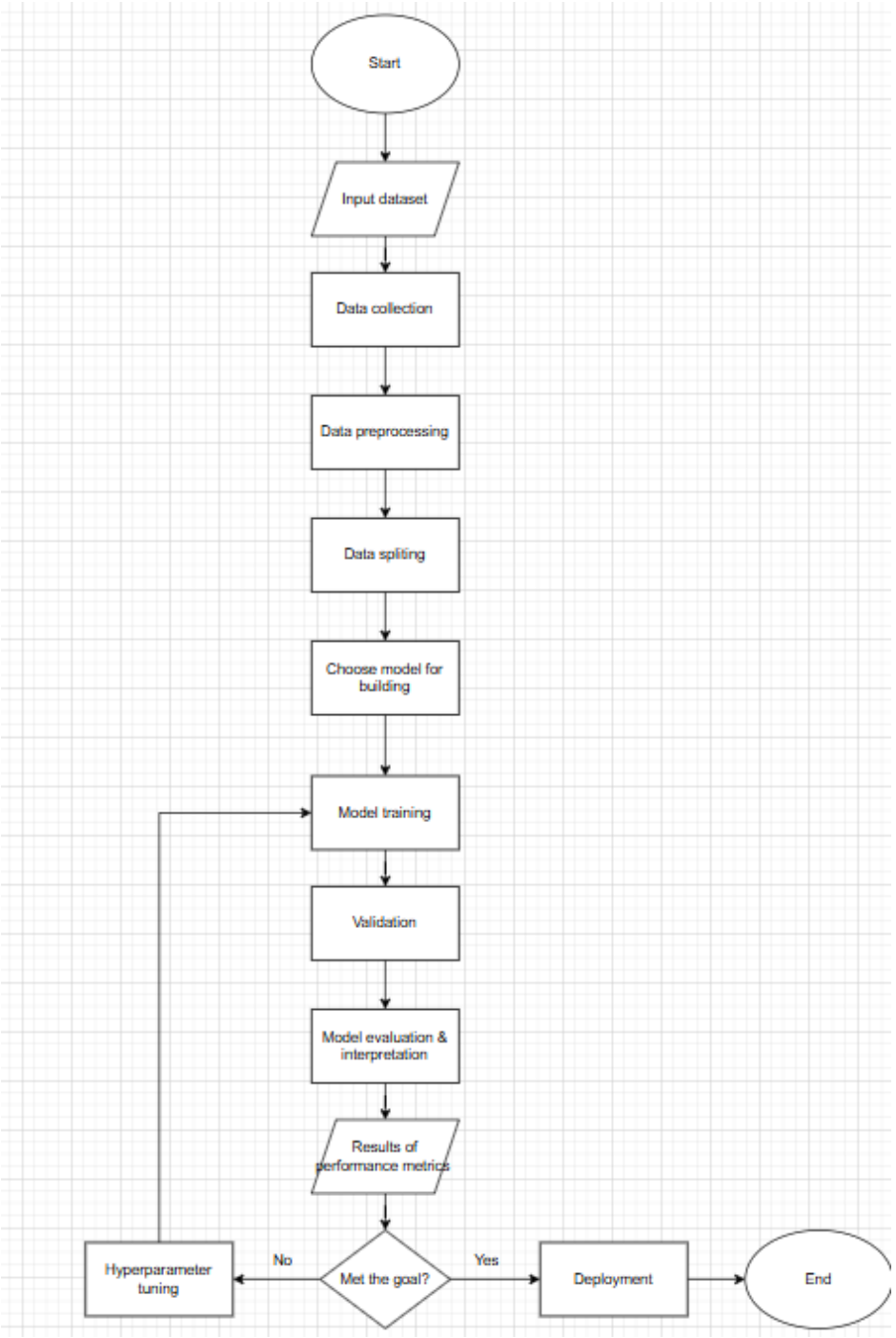
How does Deep Learning help?

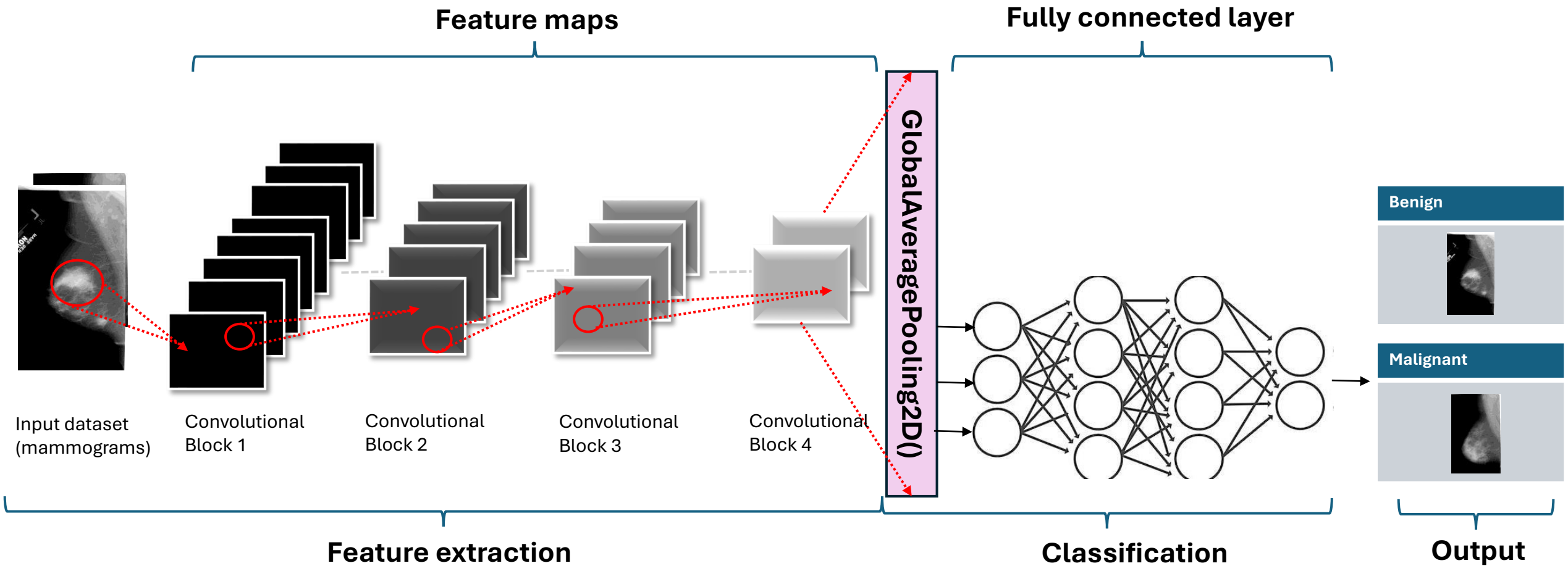


With DL

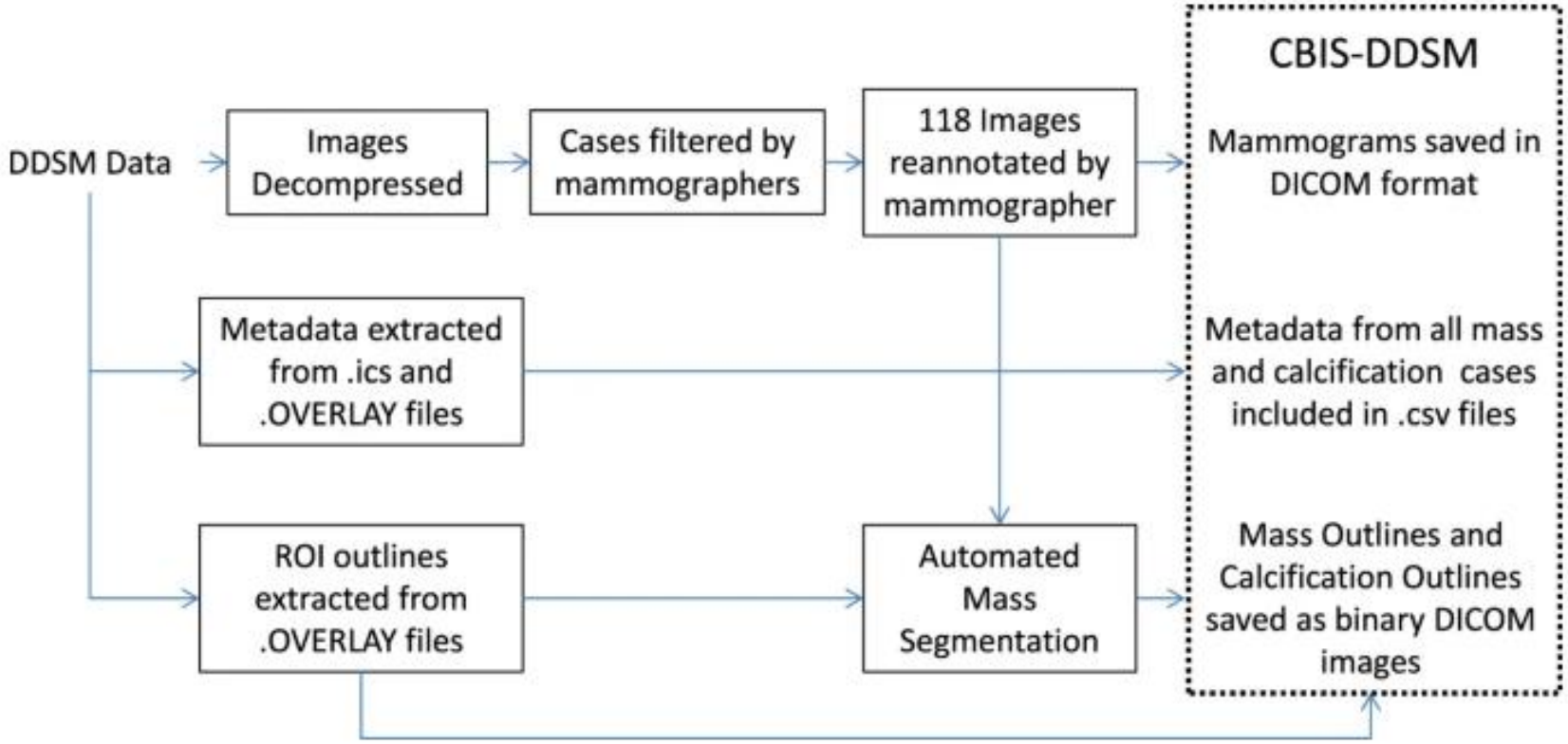
Project Approach

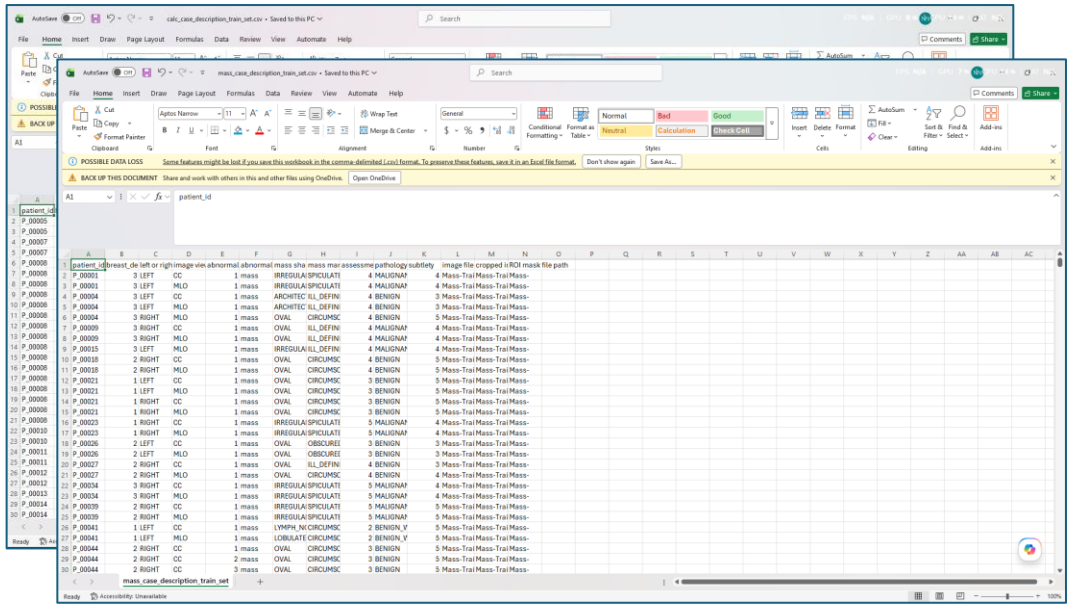
Create a Convolutional Neural Network (CNN) statistical model that has statistical power and the capability to train the Digital Database for Screening Mammography (DDSM) dataset to perform predictions and evaluate the model's predictive performance using appropriate classification metrics.




Feature maps
Fully connected layer
GlobalAveragePooling2D()
Benign
Malignant
Input dataset
(mammograms)
Convolutional
Block 1
Convolutional
Block 2
Convolutional
Block 3
Convolutional
Block 4
Feature extraction
Classification
Output

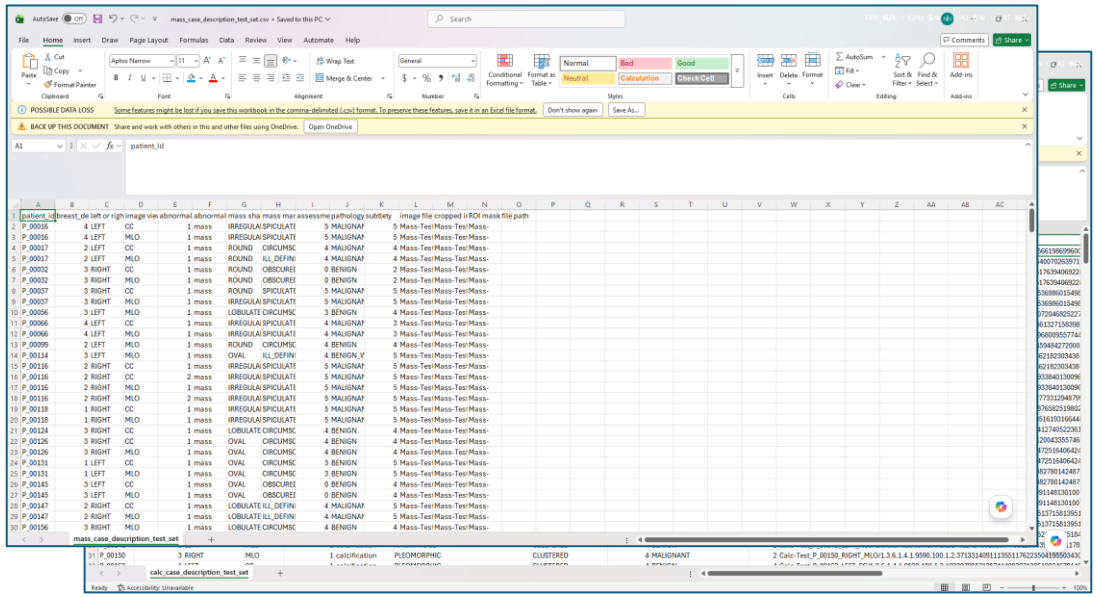
Dataset – CBIS-DDSM





calc_case_description_train.csv &
mass_case_description_train.csv

calc_case_description_test.csv &
mass_case_description_test.csv



What to expect for final submission?

- Data strategy (data augmentation) – by increasing variety of the training dataset and improve the model's ability to generalise unseen mammogram data
- Regularisation – for reducing overfitting using techniques such as dropout and weight regularisation
- Hyperparameter – by tuning the hyperparameter to fid the best parameter combination that can optimise the model's performance (e.g. batch size, learning rate, number of filters, etc)
- Transfer learning – one of the CNN frameworks listed in the resource that can address the limitations of the model

Evaluation plan

- **AUC** – ability to measure the model's diagnostic or distinguish power between the two classes (Benign, Maglinant) across all possible probability thresholds. 1.0 AUC score is the benchmark for optimal performing model
  - **Formula:** $AUC = \int_0^1 T\,PR\left(FPR^{-1}(t)\right)dt$
    Where *True Positive Rate (TRP) = True Positive/(True Positive + False Negative)*
    Where *False Positive Rate (FPR) = False Positive/(False Positive + True Negative)*
- **F1 score** – for calculating and handling class imbalance, by penalising extreme gap between precision and recall
  - **Formula**: $F_1\ \text{Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
    Where *Precision = True Positive/(True Positive + False Positive)*

Where *Recall = True Positive/(True Positive + False Negative)*

- **Sensitivity/Recall** – measures the model's ability to correctly classify actual true positive cases with minimised false negatives
  - **Formula**: Sensitivity = *True Positive/(True Positive + False Negative)*

Prototype

Results

The loss graph shows a general decline in both training and validation losses across epochs, and some instability in between, suggesting that the model is still in the learning stage. The accuracy graph shows a gradual improvement in the accuracy score peaking at

0.58, which represents a modest result that a prediction is likely to achieve around 50% accuracy. The AUC graph shows a positive uptrend that the model has decent ability to distinguish the mass and calcification cases.

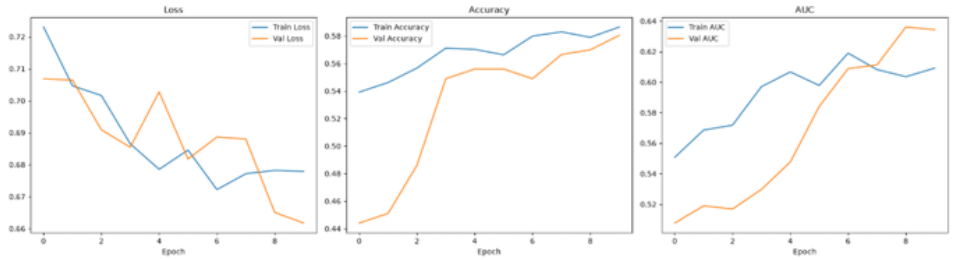


Figure 11 - Results of the CNN model (Appendix D)

Overall, the CNN model shows early signs of learning potential but requires further improvement to achieve the optimal performance to achieve the primary goal of classifying breast cancer types.

Diagrams for final submission:



[1] Define problem statement

Defining the problem statement (PS) helps to break down the questions of:

- What are you trying to solve with a machine learning model
- How are you planning to use the models,
- How will it address the problem statement
- What kind of machine learning task can be mapped to this problem

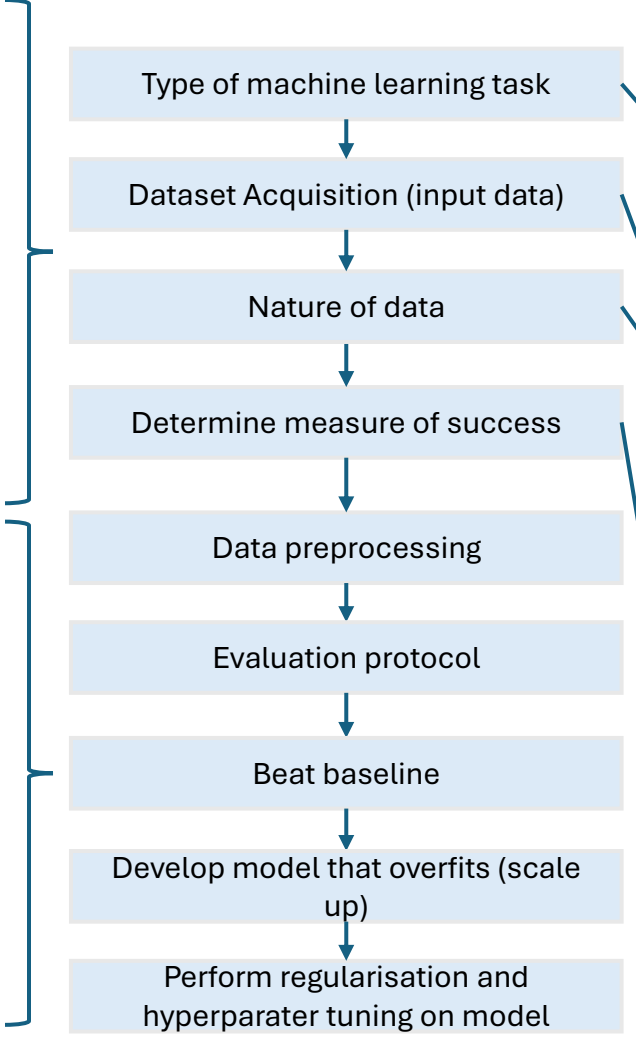
[2] Develop the model

The steps necessary to build the machine learning model to address the problem statement

[3] Deployment

In this stage the model is expected to be deployed for live demo. Typically for this stage it's expected to put the project on production environment to turn them into actual products, which can be either in the form of a device or using REST API

For this project specifically, we will be deploying this project locally



Since the PS is about determining if DL assisted X-ray mammography can improve the accuracy of breast cancer screening, by predicting whether the breast tumours in question are benign or malignant, we can identify this as a "**binary classification**" task where each input data (mammogram) should
be categorized into two exclusive categories *Benign* or *Malignant*

Digital Database for Screening Mammography (DDSM) dataset that consist of mammogram images of the breast organ, in JPEG format

Helps to gauge where the model is performing and the modelling decisions to make that can optimise their performance. For a balanced classification problem that our project falls under, accuracy and F1-Score are the one primary metrics we can use for the measurement (Chollet, 2021, p.160) for all the models listed in "Project Approach", which is dependent on the classes' imbalance ratio, supported by Macro Precision, Recall and F1-Score for the final evaluation

Formula of Accuracy: Correct Predictions *(True Positive + True Negative)* / Total Predictions *(True Positive + True Negative + False Positive + False Negative)*

Formula of Precision: True Positive / True Positive + False Positive

Formula of Recall: True Positive / True Positive + False Negative

Formula of F1-Score: 2 * (Precision * Recall) / (Precision + Recall)

Confusion Matrix as the secondary measurement is chosen to illustrate the model's predictions against actual class labels by assessing the performance of the classification across all 10 categories.

The universal machine learning workflow adapted from Deep Learning With Python (Chollet, 2021, p.154-165)

[1] Define problem statement

Defining the problem statement (PS) helps to break down the questions of:

- What are you trying to solve with a machine learning model
- How are you planning to use the models,
- How will it address the problem statement
- What kind of machine learning task can be mapped to this problem

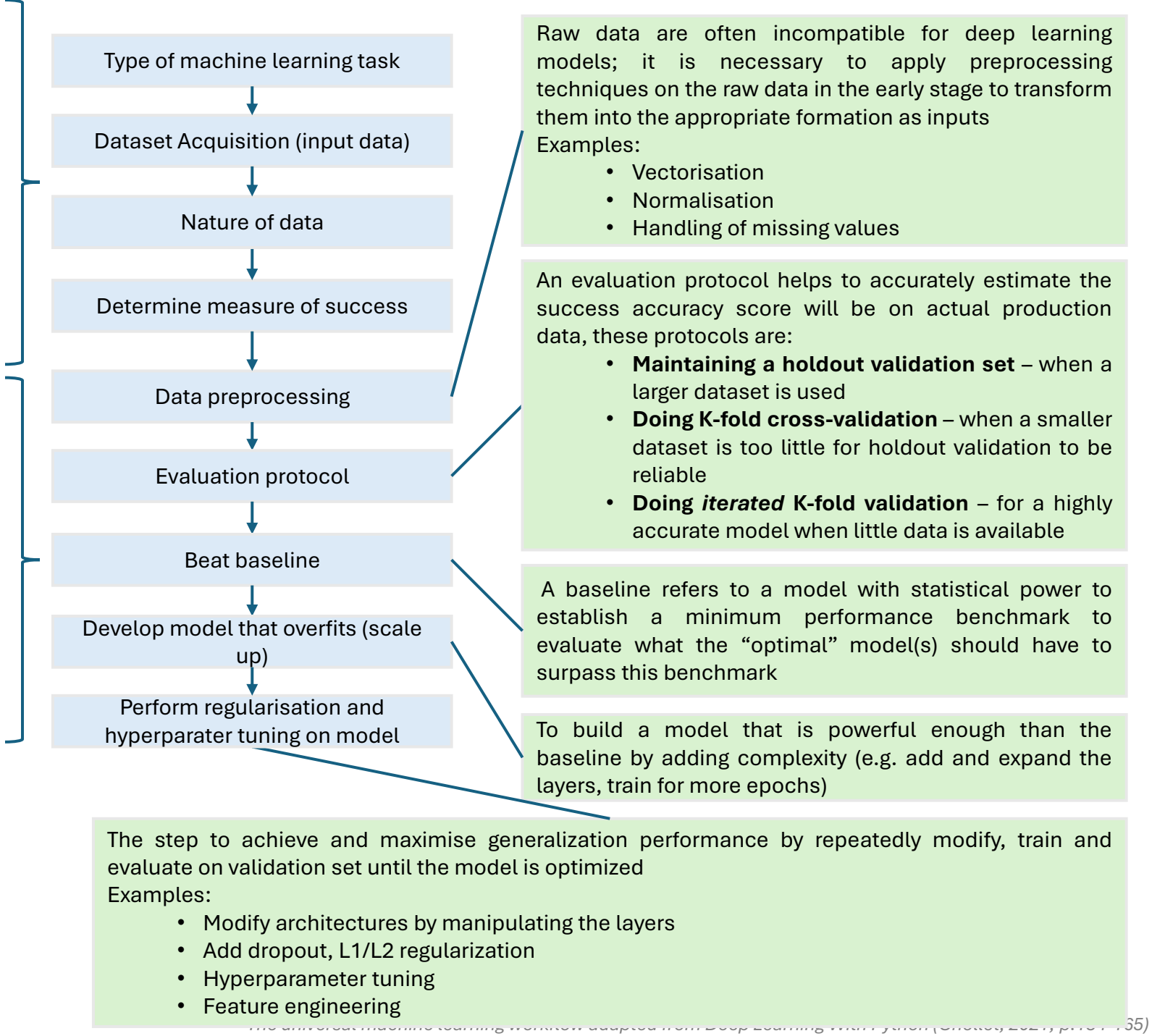
[2] Develop the model

The steps necessary to build the machine learning model to address the problem statement

[3] Deployment

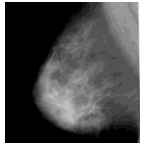
In this stage the model is expected to be deployed for live demo. Typically for this stage it's expected to put the project on production environment to turn them into actual products, which can be either in the form of a device or using REST API

For this project specifically, we will be deploying this project locally



The universal machine learning workflow adapted from Deep Learning With Python (Chollet, 2021, p.134-135)

Input layer ‘X’



X = image data
Input shape: 224 x 224 x 1 for grayscale single channel

Conv2d layer

Convolutional & activation layer | feature extraction from input
filters=32, kernel_size=3, activation="relu"

MaxPooling2d layer

Pooling layer | for reducing spatial dimensions of spatial dimensions to prevent overfitting
pool_size=2

Conv2d layer

Convolutional & activation layer
filters=64, kernel_size=3, activation="relu"

MaxPooling2d layer

Pooling layer
pool_size=2

Conv2d layer

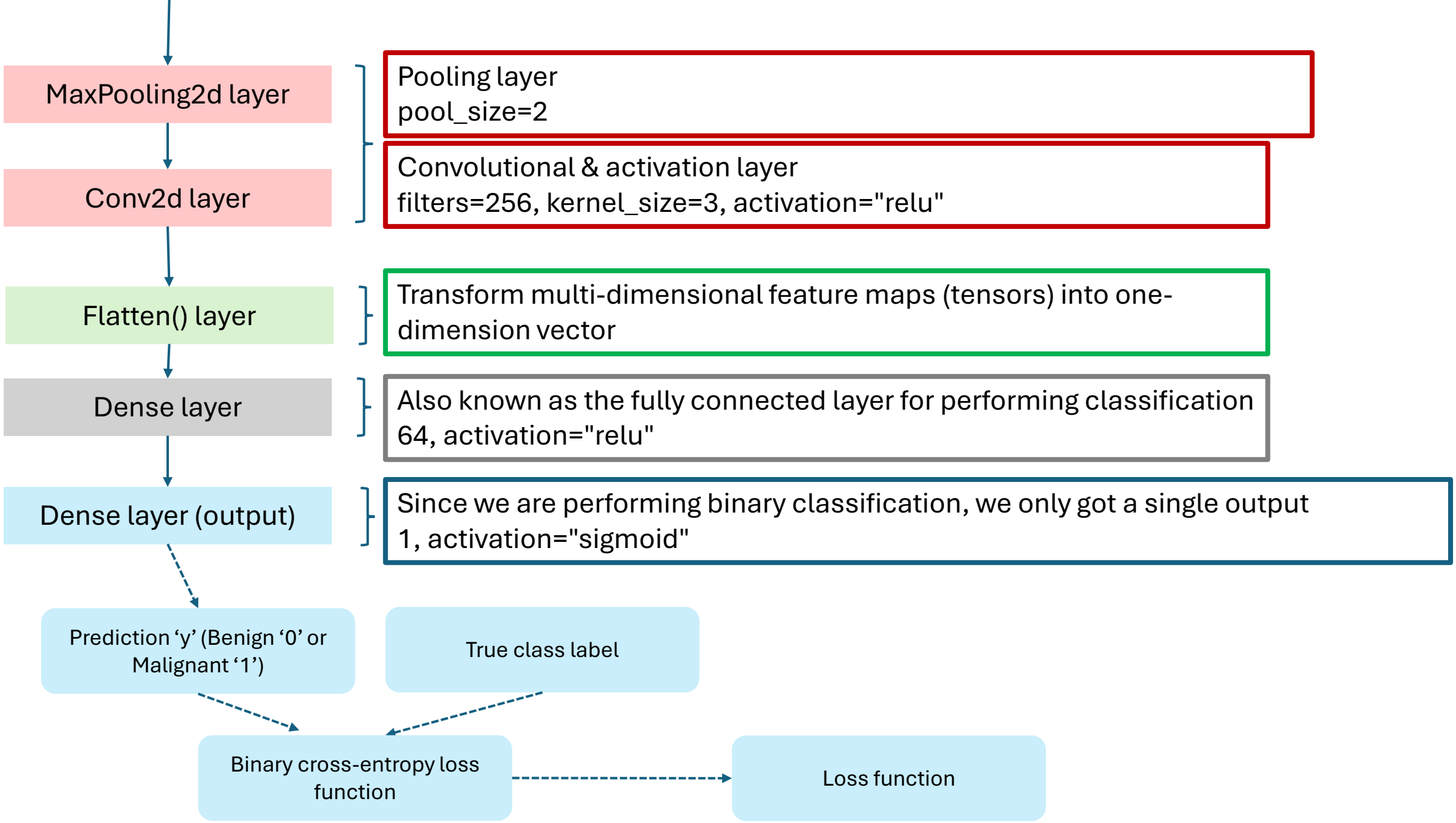
Convolutional & activation layer
filters=128, kernel_size=3, activation="relu"

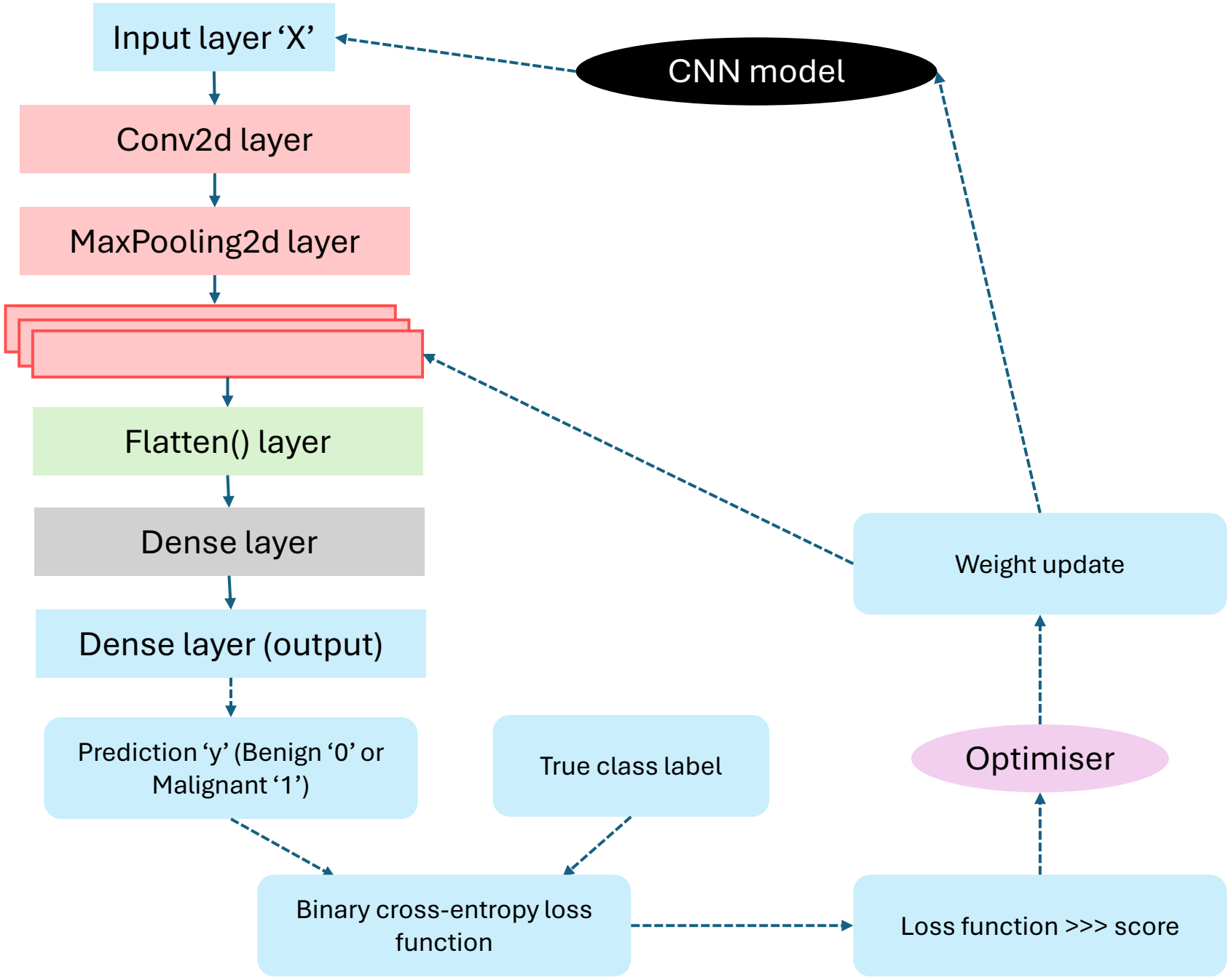
MaxPooling2d layer

Pooling layer
pool_size=2

Conv2d layer

Convolutional & activation layer
filters=256, kernel_size=3, activation="relu"


MaxPooling2d layer
Pooling layer
pool_size=2
Conv2d layer
Convolutional & activation layer
filters=256, kernel_size=3, activation="relu"
Flatten() layer
Transform multi-dimensional feature maps (tensors) into one-dimension vector
Dense layer
Also known as the fully connected layer for performing classification
64, activation="relu"
Dense layer (output)
Since we are performing binary classification, we only got a single output
1, activation="sigmoid"
Prediction 'y' (Benign '0' or Malignant '1')
True class label
Binary cross-entropy loss function
Loss function



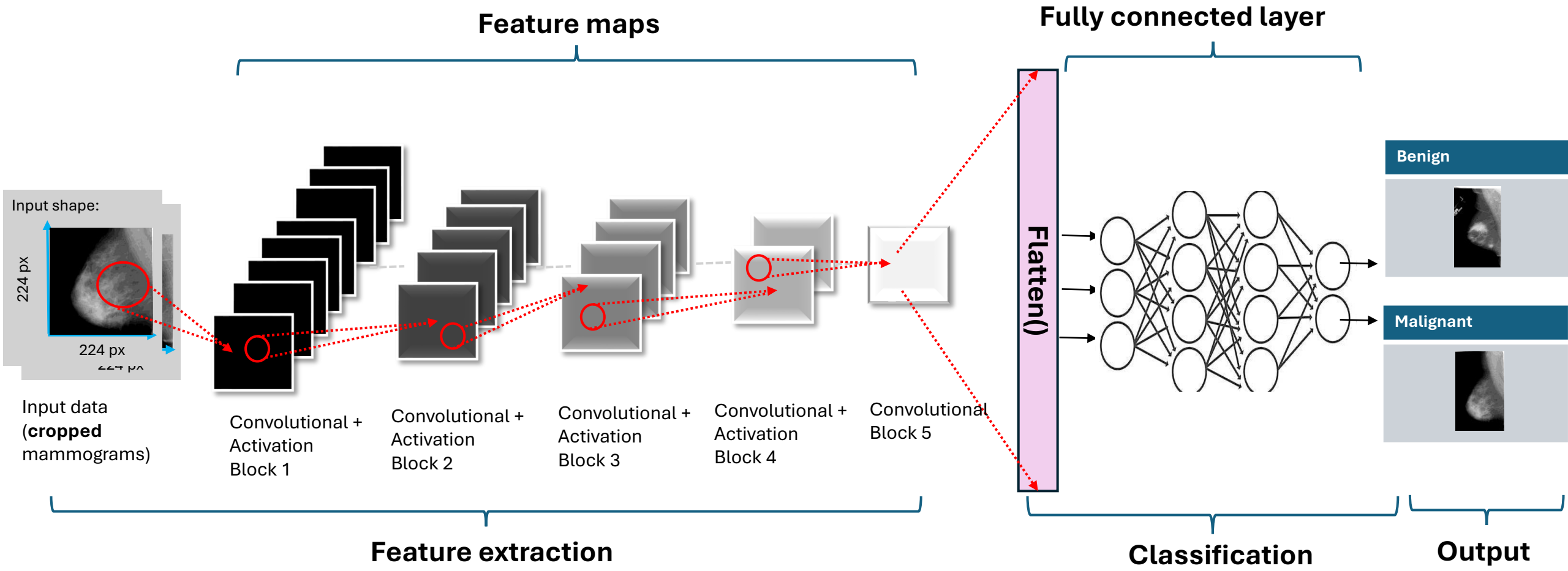
Model: "base_cnn"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 1)	0
conv2d (Conv2D)	(None, 222, 222, 32)	320
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
conv2d_3 (Conv2D)	(None, 24, 24, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_4 (Conv2D)	(None, 10, 10, 256)	590,080
flatten (Flatten)	(None, 25600)	0
dense (Dense)	(None, 64)	1,638,464
dense_1 (Dense)	(None, 1)	65

Total params: 2,616,449 (9.98 MB)

Trainable params: 2,616,449 (9.98 MB)

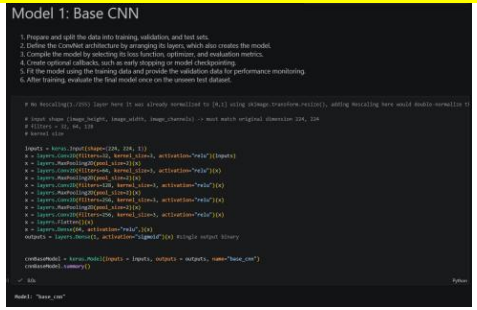
Non-trainable params: 0 (0.00 B)



CNN architecture workflow

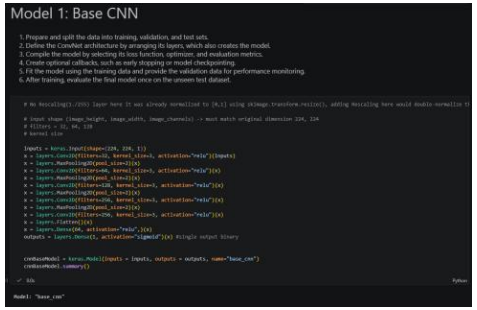
Evaluation Protocol – custom CNN models

Model 1: Base CNN



Iteration 1: Base

Model 1: Base CNN



+

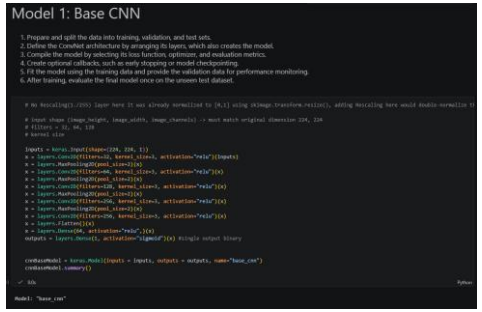
Model 2: Data Augmentation on base

```
# data_augmentation = keras.Sequential([
#     layers.RandomFlip("horizontal"),
#     layers.RandomRotation(0.01),
#     layers.RandomZoom(0.2),
# ])


data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal", seed=seed),
    layers.RandomRotation(factor=0.025, fill_mode="reflect", seed=seed + 1),
    layers.RandomTranslation(height_factor=0.04, width_factor=0.04, fill_mode="reflect", seed=seed + 2),
    layers.RandomZoom(height_factor=0.08, width_factor=0.08, fill_mode="reflect", seed=seed + 3,),
    layers.RandomContrast(factor=0.08, seed=seed + 4),
],name="mammogram_augmentation",)
```

Iteration 2: Base + data augmentation

Model 1: Base CNN



+

Model 2: Data Augmentation on base

```
# data_augmentation = keras.Sequential([
#     layers.RandomFlip("horizontal"),
#     layers.RandomRotation(0.01),
#     layers.RandomZoom(0.2),
# ])


data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal", seed=seed),
    layers.RandomRotation(factor=0.025, fill_mode="reflect", seed=seed + 1),
    layers.RandomTranslation(height_factor=0.04, width_factor=0.04, fill_mode="reflect", seed=seed + 2),
    layers.RandomZoom(height_factor=0.08, width_factor=0.08, fill_mode="reflect", seed=seed + 3,),
    layers.RandomContrast(factor=0.08, seed=seed + 4),
],name="mammogram_augmentation",)
```

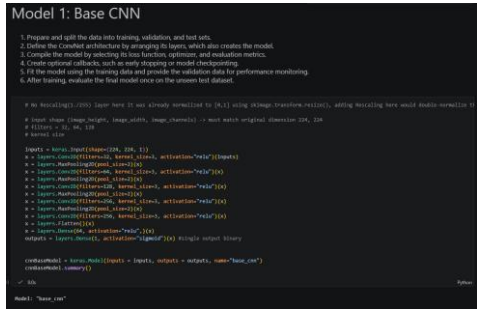
+

Iteration 3: Base + data augmentation + L2 regularisation

Model 3: L2 & Regularisation on model 2

```
# Small L2 regularisation value because Model 2 is already learning slowly
l2_reg = keras.regularizers.l2(0.00001)
```

Model 1: Base CNN



+

Model 2: Data Augmentation on base

```
# data_augmentation = keras.Sequential([
#     layers.RandomFlip("horizontal"),
#     layers.RandomRotation(0.01),
#     layers.RandomZoom(0.2),
# ])


data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal", seed=seed),
    layers.RandomRotation(factor=0.025, fill_mode="reflect", seed=seed + 1),
    layers.RandomTranslation(height_factor=0.04, width_factor=0.04, fill_mode="reflect", seed=seed + 2),
    layers.RandomZoom(height_factor=0.08, width_factor=0.08, fill_mode="reflect", seed=seed + 3,),
    layers.RandomContrast(factor=0.08, seed=seed + 4),
],name="mammogram_augmentation",)
```

+

Iteration 4: Base + data augmentation + hyperparameter tuning

Model 4: Hyperparameter tuning

```
def hpTunedCNN(hp):
    keras.backend.clear_session()

    # Hyperparameters to search.
    dense_units = hp.Choice("dense_units", values=[32, 64, 128])

    dropout_rate = hp.Choice("dropout_rate", values=[0.25, 0.40, 0.50])

    l2_strength = hp.Choice("l2_strength", values=[0.0, 1e-5, 1e-4])

    learning_rate = hp.Choice("learning_rate", values=[1e-4, 3e-4, 1e-3])

    l2_regularizer = keras.regularizers.l2(l2_strength)
```

Evaluation Protocol – pre-trained CNN models

Resnet50

```
from keras.applications.resnet50 import ResNet50
from keras.applications.resnet50 import preprocess_input, decode_predictions

# function for building the ResNet50 architecture

def baseResNet50():

    baseRN50 = keras.applications.ResNet50(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3),
        name="base resnet50",
```

ResNet50

Iteration 1: Base (with augmented data layer)

Resnet50

```
from keras.applications.resnet50 import ResNet50
from keras.applications.resnet50 import preprocess_input, decode_predictions

# function for building the ResNet50 architecture

def baseResNet50():

    baseRN50 = keras.applications.ResNet50(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3),
        name="base_resnet50",
    )
```

ResNet50

+

```
inputs = keras.Input(shape=(224, 224, 1))

x = data_augmentation(inputs)
x = layers.Concatenate(axis=-1)([x, x, x])
x = layers.Rescaling(255.0)(x)
x = preprocess_input(x)
x = resnetBase(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(units=bestHp.get("dense_units"), activation="relu", kernel_regularizer=keras.regularizers.l2(bestHp.get("l2_strength")))(x)
x = layers.Dropout(bestHp.get("dropout_rate"))(x)
outputs = layers.Dense(1, activation="sigmoid",)(x)

resnet_hpModel = keras.Model(inputs=inputs, outputs=outputs, name="resnet50_op_hyperparameters",)

print("ResNet50 trainable:", resnetBase.trainable)

resnet_hpModel.summary()

resnet_hpModel.compile(optimizer=keras.optimizers.RMSprop(learning_rate=bestHp.get("learning_rate")),
                       loss="binary_crossentropy",
                       metrics=["accuracy"])
```

Iteration 2: Base (with augmented data layer)
+ hyperparameter tuning

VGG16

```
from keras.applications.vgg16 import VGG16
from keras.applications.vgg16 import preprocess_input as vgg16_preprocess_input
[170] ✓ 0.0s

def baseVGG16():
    baseVGG = keras.applications.vgg16.VGG16(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3),
        name="base_vgg16",
```

VGG16

Iteration 1: Base (with augmented data layer)

VGG16

```
from keras.applications.vgg16 import VGG16
from keras.applications.vgg16 import preprocess_input as vgg16_preprocess_input
[170] ✓ 0.0s

def baseVGG16():
    baseVGG = keras.applications.vgg16.VGG16(
        weights="imagenet",
        include_top=False,
        input_shape=(224, 224, 3),
        name="base_vgg16",
```

VGG16

+

```
inputs = keras.Input(shape=(224, 224, 1))
x = data_augmentation(inputs)
x = layers.Concatenate(axis=-1)([x, x, x])
# Images are currently in [0, 1]
x = layers.Rescaling(255.0)(x)
# Apply VGG16-specific preprocessing
x = vgg16_preprocess_input(x)
x = vgg16Base(x, training=False)
x = layers.Flatten()(x)
x = layers.Dense(units=bestHp.get("dense_units"), activation="relu", kernel_regularizer=keras.regularizers.l2(bestHp.get("l2_strength")))(x)
x = layers.Dropout(bestHp.get("dropout_rate"))(x)
outputs = layers.Dense(1, activation="sigmoid")(x)

vgg16_hpModel = keras.Model(inputs=inputs, outputs=outputs, name="vgg16_op_hyperparameters")

print("VGG16 trainable:", vgg16Base.trainable)

vgg16_hpModel.summary()

vgg16_hpModel.compile(optimizer=keras.optimizers.RMSprop(learning_rate=bestHp.get("learning_rate")),
                      loss="binary_crossentropy",
                      metrics=["accuracy"])
[168] ✓ 0.0s
VGG16 trainable: True
```

Iteration 2: Base (with augmented data layer)
+ hyperparameter tuning

Conclusion

Limitations

- lack of robust infrastructure for running these deep learning algorithms

- lack of quality data used and poor planning pre-processing methods
    - there is a possibility my pre-processing method is problematic causing inconsistencies.

- wrongful selection and understanding of model's architecture or complexity

Conclusion

Our models are NOT reliable for clinical trial

BUT... DL continues to be widely adopted and acknowledged in the medical field, with strong potential to enhance diagnosis and treatment plan beyond breast cancer

And a complementary support tool that improves radiologists' efficiency and workload